# RL-Guided Core Placement for Multi-Chip DNN Accelerators

Technical Report and Codebase Guide

DNN_MAPPING Project

October 2026

**Abstract**

This report documents an implementation and experimental study of core placement for multi-chip, many-core DNN accelerators. The project is an independent reconstruction motivated by Wu et al.'s 2020 TODAES paper on reinforcement-learning-based core placement [1]. Because the original simulator and several parameters were not released, this work does not claim exact numerical reproduction. Instead, it makes all reconstruction assumptions explicit.

The system partitions DNN layers into vector-matrix-multiply (VMM) and vector-vector-accumulation (VVA) logic cores, maps them to a hierarchical multi-chip mesh, and minimizes a reconstructed bottleneck pipeline latency. The repository includes random search, fixed simulated annealing (SA), adaptive SA, DDPG, guided DDPG, masked PPO, PPO-guided SA, and contextual-bandit-guided SA. The active method is a contextual bandit that ranks legal SA proposals while fixed SA retains temperature control and Metropolis acceptance.

Current results provide positive matched-control evidence for the contextual bandit on AlexNet-FC and VGG16-FC. They do not yet prove superiority over the strongest non-learned proposal heuristic, over fixed SA on every workload, or over a broader class of accelerators. This report explains the theory, code, experimental evidence and limits needed to extend the project responsibly.

## 1 Scope and research question

The goal is to assign every DNN logic core to a distinct physical core so that the latency of the slowest pipeline stage is minimized. The reference problem comes from Wu et al., "Core Placement Optimization for Multi-chip Many-core Neural Network Systems with Reinforcement Learning," ACM TODAES 2020, DOI 10.1145/3418498 [1].

The project distinguishes three progressively stronger claims:

1. The program produces legal placements and finite costs.
2. A trained learner changes its scores and decisions relative to a frozen control.
3. Under matched mechanics and equal objective-evaluation budget, the learner improves the optimizer's final result.

Only the third is an optimizer claim. A single low best-so-far value does not prove learning: it can arise from random exploration, initialisation or an external repair heuristic.

## 2 Paper-aligned benchmark comparison

The reference paper's Figure 10 reports latency and throughput normalized to its sequential baseline (BS) [1]. A random-search (RS) reference is less sensitive to the unpublished sequential logical-core ordering. Table 1 therefore uses RS as the primary comparison. It is a bar chart

rather than a numerical table, so its values are two-significant-figure readings and must not be treated as newly measured paper data. The exact aggregate claim is that DDPG improves 50.5%, 38.4%, and 18.6% relative to BS, RS, and SA, respectively [1]. It evaluates one million placements for RS and SA and reports that DDPG passes their best results around 300k–400k placements.

Table 1: Approximate latency ratios read from Wu et al.'s Figure 10 [1], using RS as the primary reference. The final column is retained only as context.

| Workload | SA/RS | DDPG/RS | DDPG improvement | RS/BS |
|---|---|---|---|---|
| AlexNet CONV | ∼0.91 | ∼0.82 | ∼18% | ∼0.88 |
| AlexNet FC | ∼0.86 | ∼0.74 | ∼26% | ∼0.58 |
| AlexNet overall | ∼0.92 | ∼0.82 | ∼18% | ∼0.84 |
| VGG16 CONV | ∼0.60 | ∼0.48 | ∼52% | ∼0.67 |
| VGG16 FC | ∼0.70 | ∼0.53 | ∼48% | ∼0.40 |
| VGG16 overall | ∼0.60 | ∼0.48 | ∼52% | ∼0.67 |
| ResNet50 CONV | ∼0.74 | ∼0.62 | ∼38% | ∼0.86 |
| ResNet50 FC | ∼0.98 | ∼0.94 | ∼6% | ∼0.88 |
| ResNet50 overall | ∼0.74 | ∼0.60 | ∼40% | ∼0.86 |
| **Paper geometric average** | **∼0.76 (implied)** | **0.616** | **38.4%** | — |

The repository implements the paper-facing topology and workload counts, but the VVA throughput, logical ordering, per-layer grids and reference simulator are unavailable. Its numerical result is therefore an independent reconstruction, not a direct value-for-value replication. Table 2 uses the same RS-normalized form for the current AlexNet-FC calibrations. Within each row, larger reductions are better; comparisons between rows are not valid because the operating conditions differ.

Table 2: Project baseline calibration in paper-style format. All conditions use AlexNet-FC on $4 \times 4$ chips with $16 \times 16$ cores/chip and one million evaluations per search method.

| Condition | RS ($\mu$s) | Fixed SA / impr. vs RS | ASA / impr. vs RS | Interpretation |
|---|---|---|---|---|
| VVA=1 op/cycle; fixed grids; seed 0 | 23.93728 | 22.70400 / 5.15% | 23.09328 / 3.53% | Fixed SA is below the paper's ∼26% AlexNet-FC DDPG-versus-RS reading. BS = 24.92032 $\mu$s; compute floor is 89.9% of BS. |

| Condition | RS ($\mu$s) | Fixed SA / impr. vs RS | ASA / impr. vs RS | Interpretation |
|---|---|---|---|---|
| VVA=16 ops/cycle; rate-one grids held fixed; seed 0 | 3.07200 | 2.05125 / 33.23% | 2.39488 / 22.04% | Fixed SA exceeds the paper chart's ∼26% AlexNet-FC DDPG-versus-RS reading, but this is an exploratory one-seed assumed-rate result, not RL. BS = 3.92032 $\mu$s. |

The learned contextual-bandit method is deliberately excluded from Table 2: its 100,001-call focused-candidate experiments answer a different causal question than the paper's million-placement BS/RS/SA comparison. Its valid improvements are 1.965% on held-out AlexNet-FC seeds, 2.255% on VGG16-FC development seeds, 1.637% on held-out VGG16-FC seeds, and 1.946% over all six VGG16-FC seeds, always relative to the matched focused-uniform frozen control. A paper-aligned learned-SA claim requires a frozen execution-rate and logical-order condition, fixed SA/ASA/learned-SA under the same one-million-call budget, and multiple predeclared seeds.

# 3 Hardware and workload reconstruction

## 3.1 Physical topology

The paper-oriented configuration has a $4 \times 4$ array of chips. Each chip contains a $16 \times 16$ mesh of processing cores, for

$$N_{\text{physical}} = 4 \times 4 \times 16 \times 16 = 4096 \tag{1}$$

physical cores. The implementation is in `src/multi_chip_topology.py`. A global core ID is chip-major; helper functions recover chip and local coordinates.

The active paper-oriented routing model, `paper_xy`, uses dimension ordered routing. Intra-chip traffic follows XY routing in the local mesh. Inter-chip traffic routes from the source core to a lower-left chip-periphery gateway, travels X then Y across the chip array, then travels from the target gateway to the destination core. This gateway choice and detailed global routing-switch behaviour are reconstruction assumptions.

## 3.2 DNN logic cores

The workload extractor in `src/run_multi_chip.py` traces supported Torchvision models and represents every layer with a two-dimensional partition grid. For $M$ output-channel groups and $N$ input-channel groups, the layer contains $MN$ VMM cores and $M$ VVA cores. VMM tasks compute partial products; VVA tasks accumulate partial sums and emit activations.

`src/compute_model.py` implements the partition reconstruction. In `paper_targets` mode, it selects per-layer $(M, N)$ grids that match the aggregate logic-core totals reported in the reference paper [1], satisfy the 64 KB 8-bit weight-tile restriction, and balance estimated VMM/VVA work.

The paper does not publish individual layer grids, VVA execution rate or the logical indexing order. These settings are documented reconstruction parameters, not recovered ground truth.

Table 3: Published aggregate logic-core targets used by the reconstruction [1].

| Model | CONV logic cores | FC logic cores |
|---|---|---|
| AlexNet | 183 | 932 |
| VGG16 | 1,024 | 1,924 |
| ResNet50 | 512 | 37 |

### 3.3 Compute timing

VMM timing uses 128 MACs per cycle at a 400 MHz clock. VVA timing uses the explicit parameter `--vva_ops_per_cycle`, because the paper does not publish that rate. For task $i$,

$$t_{\text{comp}}(i) = \frac{\lceil \text{operations}_i / (\text{rate}_i \, u) \rceil}{400 \text{ MHz}}, \tag{2}$$

where $u$ is assumed arithmetic utilization. The model excludes several unpublished factors, including detailed packet scheduling, activation-buffer stalls, router startup and exact compute/communication overlap.

## 4 Placement objective and evaluator

### 4.1 Legal placement

Let $P(i)$ be the physical core assigned to logic task $i$. A placement is legal when every assignment is in the allowed physical-core region and $P(i) \neq P(j)$ for $i \neq j$. `MultiChipEnvironment` in `src/multi_chip_environment.py` owns this state, validates legality and calculates cost.

The code can allocate disjoint whole-chip masks to convolutional and fully-connected regions, matching the paper's high-level separation of these workload classes.

### 4.2 Bottleneck pipeline latency

The primary objective is reconstructed bottleneck pipeline latency, not summed wirelength. Kahn topological levels form task stages $S_k$. The objective is

$$L(P) = \max_k T_k(P). \tag{3}$$

In the basic task-sum model, a placed source task has service time

$$t_i(P) = t_{\text{comp}}(i) + \sum_{j \in \text{succ}(i)} \text{bytes}(i, j) \, c(P(i), P(j)), \tag{4}$$

and stage time is the largest service time in that stage. Under `paper_pipeline`, the evaluator constructs explicit XY routes and models shared directed-link serialization. For each phase it uses the larger of the largest single-source routed traffic time and the most loaded directed link.

The methods `pipeline_stages()`, `pipeline_breakdown()` and `_xy_contention_components()` implement this calculation.

### 4.3 Why sensitivity is checked first

An optimizer cannot reliably improve a nearly flat objective. Earlier AlexNet CONV partitioning concentrated work in a placement-independent VVA task, creating an apparent optimization problem with almost no real headroom. The current runner samples random legal placements before long paper-mode runs and can stop if the observed relative objective span is too small. The bypass flag `--allow_flat_objective` is intended for diagnostics, not normal experiments.

### 4.4 Incremental exact evaluation

SA moves relocate or swap only a few tasks. `IncrementalPipelineEvaluator` in `src/run_multi_chip.py` caches unaffected stage costs and evaluates only stages affected by a candidate move. It is designed to agree with complete evaluation while allowing large SA budgets. Full-evaluation flags remain available for debugging and reference checks.

## 5 Classical search baselines

Table 4: Implemented optimization methods.

| Method | CLI value | Role |
|---|---|---|
| Sequential baseline | `bs` | Deterministic chip-major reference placement. |
| Random search | `random` | Independent valid placements; retain the best sampled cost. |
| Fixed SA | `sa` | Primary classical baseline with calibrated geometric cooling. |
| Adaptive SA | `asa` | Temperature and neighborhood feedback baseline. |
| Continuous DDPG | `ddpg` | Historical paper-oriented actor–critic path. |
| Guided DDPG | `ddpg_guided` | Legal-action DDPG with demonstrations. |
| Masked PPO | `ppo_masked` | Constructive legal categorical placement policy. |
| Learned SA | `ppo_asa` | PPO or contextual-bandit proposal selection inside SA. |

For fixed SA, a candidate $P'$ is accepted automatically if $\Delta = L(P') - L(P) \leq 0$. Otherwise it is accepted with probability

$$p_{\text{accept}} = \exp(-\Delta/T). \tag{5}$$

The initial temperature is calibrated from uphill candidate deltas to target an initial acceptance probability, normally 0.8. Temperature then cools geometrically with the paper-oriented factor 0.99. This calibration replaced historical absolute temperatures that were inconsistent with a seconds-valued objective.

Adaptive SA adjusts temperature and move scale over windows to target an acceptance rate near 0.30. It is a useful comparison, but its current controller has generally performed worse than calibrated fixed SA on the evaluated settings. That is a result about this controller, not a general rejection of adaptive metaheuristics.

## 6 Retained reinforcement-learning methods

### 6.1 Continuous DDPG

The original paper-oriented DDPG path [1] has actor and critic networks that emit continuous coordinates and estimate action values. It supports an MLP, a spatial CNN and a Figure-9-inspired CNN. Actions are discretized and collision-repaired when several tasks map to the same physical core.

This repair causes a credit-assignment problem: replay stores the actor's continuous intent while the evaluator sees a substantially different repaired placement. Furthermore, continuous actor–critic updates are prone to deterministic sparse-reward deadlock [4]. Earlier diagnostics

showed high repair rates and degenerate intended positions. The DDPG implementation is therefore retained for historical audit, not treated as the active solution.

### 6.2 Guided DDPG and masked PPO

`run_ddpg_guided()` attempts to improve DDPG with Wolpertinger-style legal candidate ranking [5], ASA demonstrations, prioritized replay, twin critics and behaviour cloning. `run_masked_ppo()` places tasks autoregressively with an exact invalid-action mask [6, 7] over occupied and disallowed cores, eliminating collision repairs. Its PPO objective is

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]. \tag{6}$$

These methods demonstrate legal-action mechanics (and masked PPO demonstrated positive policy learning against its frozen control on single-chip AlexNet-CONV) but have not provided a robust final-quality gain over strong SA baselines on multi-chip workloads in the documented experiments.

### 6.3 PPO-guided SA

The PPO-SA path ranks a dynamic set of legal neighbor moves while SA retains candidate generation and Metropolis acceptance [2, 3]. The `reference_separate` update style uses disjoint actor and critic modules, separate optimizers, reward-to-go value targets and GAE-style advantages. It was informed by the independently adapted training structure of the public RL-Based-SA project [2], whose attribution is recorded in `THIRD_PARTY_NOTICES.md`. Long tests did not establish a reliable PPO proposal advantage, motivating a simpler one-step learner.

## 7 Active method: contextual-bandit-guided fixed SA

### 7.1 Design

The active method retains fixed SA's global optimization mechanics and learns only to choose a move from a legal candidate set. This is a contextual-bandit formulation because one objective evaluation reveals the selected proposal's immediate gain. It avoids using a long-horizon value model when the supervised signal is naturally one-step.

At each SA step, the system creates $K = 16$ legal candidate moves. In the active configuration, half are anchored on current bottleneck-stage tasks and half are ordinary legal proposals. SA remains responsible for initial placement, candidate legality, temperature, uphill-move acceptance and best-so-far retention.

### 7.2 Candidate representation

Each proposal receives nine features:

1. bottleneck-stage membership;
2. VVA-task indicator;
3. normalized task work;
4. task work fraction within its stage;
5. predecessor-hop change;
6. successor-hop change;

7. traffic-weighted hop change;

8. same-chip connectivity change;

9. an inexpensive routed-communication `proxy_gain`.

The global context contains chain progress, previous objective delta, previous accept/reject state and recent acceptance fraction.

### 7.3 Bandit network and update

`ProposalBanditRegressor` uses a global-state encoder and candidate encoder, each a one-layer 128-unit `tanh` MLP. Their embeddings are added, nonlinearly transformed and passed to a scalar score head. The score head is initialised to zero. Before learning, all candidates have equal score, which makes the frozen control exactly uniform.

For selected proposal $P'$, the normalized signed proposed reward is

$$r = \mathrm{clip}\left(\frac{L(P) - L(P')}{s}, -10, 10\right), \tag{7}$$

where $s$ is the reward scale. Positive reward means the proposal improved the objective before the Metropolis accept/reject decision. The network minimizes smooth-L1 loss,

$$\mathcal{L}_{\mathrm{bandit}} = \mathrm{Huber}\left(\hat{r}(g, x), r\right), \tag{8}$$

using uniform replay and AdamW. Uniform replay is deliberate: an earlier positive-balanced replay scheme overrepresented rare gains and led to harmful overprediction.

### 7.4 Exploration and controls

After a uniform warm-up, epsilon-greedy selection gradually reduces random selection to 0.50 across the entire run. This relatively high final exploration rate is intentional because only selected candidates obtain labels.

The primary frozen control has the same candidate pool, warm-up, cooling, acceptance rule, budget and random-stream structure, but makes no parameter updates. A stricter `proxy_greedy` control ranks candidates directly by the hand-designed `proxy_gain` feature; it tests whether the learned model contributes beyond that heuristic.

## 8 Current evidence

The supported conclusion is deliberately narrow:

> Under equal true-objective budgets and the same focused candidate pool, the contextual-bandit selector improved over a focused-uniform frozen control on tested AlexNet-FC and VGG16-FC workloads. It is not yet proven superior to the best non-learned proposal heuristic, fixed SA across every workload, or a broad class of accelerators.

Note that on FC workloads, restricting proposals to a 50% bottleneck-anchored 16-candidate pool causes the frozen focused-uniform control to finish slightly behind unrestricted plain fixed SA; the bandit's $\sim$2% gain over its paired frozen control recovers that gap, resulting in an approximate tie with plain fixed SA.

The proxy-greedy comparator is the immediate next causal test. It holds the candidate set and objective budget fixed while removing training. A longer matched proxy-greedy study is required before claiming that the learned scorer adds value beyond the hand-designed heuristic.

Table 5: Selected recorded evidence (100,001 objective evaluations per run on FC workloads). Detailed per-seed costs and confidence intervals are documented in `docs/PROJECT_STATE.md`.

| Protocol | Main observation | Interpretation |
|---|---|---|
| AlexNet-FC, held-out seeds 3–5 | Learned beat frozen control 3/3 (mean 2.20656 $\mu$s vs. 2.25077 $\mu$s; +1.965%, 95% CI [1.008%, 2.922%]). Fixed SA mean 2.22160 $\mu$s; ASA 2.43248 $\mu$s. | Positive held-out-seed evidence on a graph used in development; not a new-instance test. |
| AlexNet-CONV, 3 seeds | Learned, frozen and fixed-SA conditions reached the same final minimum. | Valid final-quality tie on a single-chip region. |
| VGG16-FC, seeds 0–2 | Learned beat frozen control 3/3; mean relative improvement +2.255%. | Positive transfer signal without retuning hyperparameters. |
| VGG16-FC, held-out seeds 3–5 | Learned beat focused-uniform 3/3 (+1.637% mean); effectively tied with fixed SA. | Strengthens the control result, but does not prove fixed-SA superiority. |
| VGG16-FC, all 6 seeds | Learned beat focused-uniform 6/6; mean relative improvement +1.946%. | Strongest current cross-workload evidence, still exploratory. |

# 9 Limitations and threats to validity

## 9.1 Evaluator fidelity

The implementation is a documented sensitivity model, not the unpublished reference simulator. The main gaps are exact global routing, VVA throughput, activation-buffer stalls, detailed packet scheduling, router startup, compute/communication overlap and logical task ordering. Paper-normalized ratios should therefore be reported as reconstruction diagnostics rather than exact reproduction proof.

## 9.2 Causal attribution

The frozen focused-uniform control isolates the package of trained scoring plus the selected candidate representation. It does not isolate the learned model from bottleneck focusing or `proxy_gain`. The proxy-greedy control, feature removal, focus-zero and shuffled-label studies are the appropriate next ablations.

## 9.3 Statistics and wall-clock cost

Seeds are repeated stochastic executions, not independent workload instances. Three-seed uncertainty intervals are descriptive and unstable. Every report should provide individual seeds, complete-objective-call budgets, wall-clock time and predeclared decision rules. Equal objective-call counts do not imply equal wall-clock cost because learned SA constructs, featurizes and scores a candidate pool and performs neural updates.

# 10 Codebase guide

Table 6: Active repository components.

| Location | Responsibility | Read it when you need to |
|---|---|---|
| `src/run_multi_chip.py` | Workload extraction, algorithms, neural models, CLI and reports. | Change an algorithm, a reward, a candidate feature or a command-line option. |
| `src/multi_chip_environment.py` | Legality, Kahn stages, objective and diagnostics. | Change or validate latency semantics. |
| `src/multi_chip_topology.py` | Physical IDs, mesh/torus distances and XY routes. | Change topology or routing construction. |
| `src/compute_model.py` | Layer grids, operation counts and timing. | Change partition or VMM/VVA assumptions. |
| `src/run_paper_experiment.py` | CONV/FC paper-style orchestration. | Run a documented reproduction suite. |
| `src/run_multiseed_experiment.py` | Independent multi-seed scheduler. | Run matched repeated experiments. |
| `src/analysis/` | Post-hoc analysis and gates. | Analyse an existing archive without changing optimization. |
| `src/experiments/` | Historical and exploratory prototypes. | Inspect old approaches; do not treat them as supported runners. |
| `scripts/` | Kaggle and batch experiment gates. | Run preserved protocols. |
| `tests/` | Smoke and reconciliation tests. | Verify a change before a long experiment. |
| `docs/PROJECT_STATE.md` | Current quantitative evidence. | Make a result claim. |
| `docs/RECONCILIATION.md` | Paper assumptions and evaluator corrections. | Compare the implementation with the source paper. |
| `docs/RESEARCH_FINDINGS.md` | Literature, failure analysis and causal concerns. | Choose the next experiment. |
| `docs/PROJECT_JOURNEY.md` | Chronological project history. | Understand why prior decisions changed. |
| `docs/archive/` | Preserved superseded evidence. | Audit older claims. |

# 11 Reproducibility protocol

For every reported experiment:

1. Record Git commit, model, workload region, topology, timing model, VVA rate, logical task order, seed and device.
2. Run the objective-sensitivity preflight before large paper-mode search.
3. Use matched complete-placement objective evaluations.
4. Keep learning and frozen-control conditions identical except for parameter updates.
5. Save configuration/report JSON, diagnostic JSONL, manifest and archive checksum.
6. Report every seed and distinguish tuned, held-out and new-workload experiments.

The active family is selected with `--algo ppo_asa`, `--ppo_asa_learner bandit`, and `--ppo_asa_annealing fixed`. Exact supported commands belong in the repository README and `docs/IMP_README.md`; experiment commands should not be reconstructed from memory.

## 12 Publication path

The present result is suitable for a bounded workshop or reproducibility-style submission only if claims remain matched to evidence. A stronger positive-method paper needs:

1. a longer proxy-greedy matched-control experiment;
2. feature and candidate-focus ablations;
3. more held-out seeds and at least one additional workload family;
4. a frozen evaluator configuration and recoverable archives; and
5. explicit reconstruction limits in the method and discussion sections.

The defensible contribution is not that RL universally beats SA. It is a contextual-bandit candidate-selection mechanism for fixed SA in a reconstructed multi-chip DNN placement simulator, with matched controls, documented negative results and a reproducible experimental methodology.

## References


[1] N. Wu, F. Deng, Y. Xie, et al. Core Placement Optimization for Multi-chip Many-core Neural Network Systems with Reinforcement Learning. *ACM Transactions on Design Automation of Electronic Systems*, 2020. DOI: 10.1145/3418498.

[2] N. Qiu and R. Liang. Reinforcement-Learning-Based Simulated Annealing. In *Proceedings of AAMAS*, 2025. Public implementation audited by this project: https://github.com/nathanqiu07/RL-Based-SA-Public.

[3] D. Vashisht et al. Placement in Integrated Circuits using Cyclic Reinforcement Learning and Simulated Annealing. In *NeurIPS Workshop on Machine Learning for Engineering Modeling, Simulation and Design*, 2020. https://arxiv.org/abs/2011.07577.

[4] G. Matheron, N. Perrin, and O. Sigaud. The Problem with DDPG: Understanding Failures in Deterministic Environments with Sparse Rewards. *arXiv preprint arXiv:1911.11679*, 2019. https://arxiv.org/abs/1911.11679.

[5] G. Dulac-Arnold et al. Deep Reinforcement Learning in Large Discrete Action Spaces. *arXiv preprint arXiv:1512.07679*, 2015. https://arxiv.org/abs/1512.07679.

[6] S. Huang and S. Ontañón. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. *arXiv preprint arXiv:2006.14171*, 2020. https://arxiv.org/abs/2006.14171.

[7] W. Myung et al. Policy Gradient-Based Core Placement Optimization for Multichip Many-Core Systems. *IEEE Transactions on Neural Networks and Learning Systems*, 2021. DOI: 10.1109/TNNLS.2021.3117878.